

Cyber Physical Robotics: Project description

July 31, 2026

1 Overview

The purpose of this project is to train you in the design of distributed and concurrent software, particularly as it related to robotic control, subject to communication challenges.

2 Implementation

A simulation must be developed in any programming language of choice. The simulation must be of a 2D grid (10x10), where a configurable number of robots move according to rules. *Targets* in the simulation must show up randomly, and persist until they are captured by robots. Each robot has a unique ID.

2.1 Robot operation

At each turn, a robot can perform one of these actions:

- Move forward one square, in the direction it is facing
- Turn left (rotate 90° counter-clockwise)
- Turn right (rotate 90° clockwise)
- Pick up target from its current position

At each turn, a robot can sense:

- If there is a target in the position directly in front
- If there are robots in the position directly in front (and what their IDs are)
- If there are robots in the same position it currently occupies (and what their IDs are)

At each turn, a robot can broadcast a message (meaning, all other robots receive it). It is up to you what the format and payload of that message is.

2.2 Simulation behavior

The simulation *must* obey the following procedure:

```
while (number iterations < max-iterations)
  if (no more targets)
    generate random targets
  for all robots
    read input messages
  for all robots
    sense environment
    decide on action to perform
  Test for illegal behavior here
  for all robots
    perform action
  for all robots
    send output messages
```

The simulation must test for two illegal conditions:

- At least one robot attempts to pick up a target from a position where there is no target
- Exactly one robot attempts to pick up a target from a position where there is a target
- More than two robots attempt to pick up a target from a position where there is a target

In other words, targets can only be picked up validly by exactly two robots, at exactly the same time.

2.3 Communication

Communication must be subject to random delay: specifically, following an exponential distribution. For each message each robot sends, all other robots will receive such a message at potentially different times: the random delay is applied per receiver, not per message. The easiest way to implement this is:

```
Procedure send-message(msg)
  for all robots
    r = generate-random-number ;exponential distribution
    input-message-queue <- append(msg, r)

Procedure read-messages()
  for all messages in input-message-queue
    if (r == 0)
      read msg
    else
      r = r-1
```

3 Deliberables

Three milestones throughout the term. At each milestone, you must submit a report describing your implementation. In the report, **no code**: high level descriptions only (flowcharts, state machines, CSPs, Petri nets...) and present your simulation.

- **Milestone 1:** No communication delay, and no full robot behavior. Simply illustrate the failure cases: i.e., show that your simulation can detect all illegal conditions.
- **Milestone 2:** No communication delay. Full robot behavior: show that, given perfect, ideal communication, your distributed control works.
- **Milestone 3:** Communication delay as described above. Show your distributed control still works despite the unreliability of communication.